

EV Charging Schedule Optimization

Technical Documentation

1. Overview

Produces an hour-by-hour EV charging plan from a vehicle spec and an hourly forecast (price, solar, plug-in confidence). Goal: reach the target state of charge (SoC) at minimum cost, preferring solar and high-confidence hours.

2. Input and Output Formats

2.1 Input

```
{
  "vehicle": {"capacity": 100, "current_soc_pct": 30,
              "target_soc_pct": 72, "max_power_kw": 9.0},
  "forecast": [
    {"hour": "6:00", "price": 0.32, "solar": 0.0, "confidence": 1.00},
    {"hour": "7:00", "price": 0.28, "solar": 0.0, "confidence": 1.00}
  ],
  "params": {"confidence_floor": 0.95}
}
```

Field reference:

- `vehicle.capacity` — total battery capacity in kWh (e.g., 100 = 100 kWh pack).
- `vehicle.current_soc_pct` — battery charge right now, as a percentage 0–100.
- `vehicle.target_soc_pct` — desired charge by the end of the horizon, percentage 0–100.
- `vehicle.max_power_kw` — maximum charging rate the vehicle/charger supports, in kW.
- `forecast[].hour` — timestamp or hour label for that forecast slot (e.g., "6:00").
- `forecast[].price` — grid electricity price for that hour, in €/kWh.
- `forecast[].solar` — available solar power for that hour, in kW.
- `forecast[].confidence` — probability (0–1) that the car will actually be plugged in during that hour.
- `params.confidence_floor` — demand buffer multiplier; lower value plans for more energy as a hedge (default 0.95).

2.2 Output

```
[
  {"hour": "2026-06-13T06:00:00Z", "charging_power": 0.0},
  {"hour": "2026-06-13T07:00:00Z", "charging_power": 9.0}
]
```

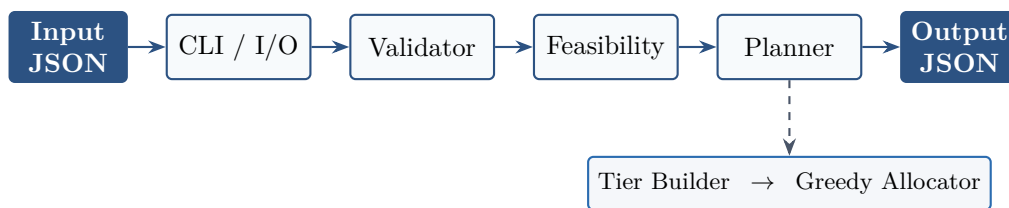
On validation failure: `{"error": [...]}`, exit code 1.

3. Architecture

Three sequential stages, orchestrated by `app/main.py`:

Stage	Module	Responsibility
1. Validate	<code>utils/data_validator.py</code>	Pydantic field-range checks; rejects malformed input.
2. Feasibility	<code>utils/feasibility_checker.py</code>	Estimates whether the target is reachable in the horizon.
3. Plan	<code>service/planner_service.py</code>	Tiered greedy allocation of kWh per hour.

Block Diagram



4. How the Schedule is Built

4.1 Energy Tiers

Each forecast hour produces up to two tiers:

- **Solar tier** — cost 0 (when free), capped at `min(solar_kW, max_power_kW)`.
- **Grid tier** — cost = hourly price, fills the remaining `max_power` capacity.

Each tier is ranked by **adjusted cost** = `raw_cost/confe`, where $e = \text{confidence_exponent}$ (default 0.5). Dividing by confidence pushes uncertain hours down; the exponent controls how aggressively.

```
weight = confidence ** confidence_exponent
tiers.append((raw_cost / weight, raw_cost, energy_kwh, hour_index))
tiers.sort(key=lambda t: t[0])
```

Example (default $e = 0.5$): hour A (price 0.20, conf 1.00) ranks at $0.20/\sqrt{1.00} = 0.200$; hour B (price 0.15, conf 0.60) ranks at $0.15/\sqrt{0.60} \approx 0.194$ — B wins. Setting $e = 1.0$ gives the hard penalty (0.20 vs. 0.25) where A wins instead.

Edge cases. Confidence $\leq 10^{-6}$ is treated as “not plugged in” and the hour goes to the bottom of the list. When `solar_is_free=True` the raw solar cost is 0, so the divisor has no effect — free solar always ranks at the top.

4.2 Greedy Allocation

- Take from cheapest tier first until total $\geq$ **buffered demand** (`energy_to_target / confidence_floor`).
- Then opportunistically top up while *raw* cost < `value_of_full` (default 0.0 — disabled).
- Stop at full battery or when no profitable tier remains.

4.3 Confidence Buffer

Demand is inflated by $1/\text{confidence_floor}$ before allocation. A 0.95 floor plans for $\sim 5\%$ extra energy as a hedge against optimistic forecasts.

5. Feasibility Check

- Convert SoC % to kWh; required = $\max(0, \text{target} - \text{current})$.
- Buffered demand = required / confidence_floor, capped at remaining physical capacity.
- Upper bound = max_power_kW × forecast_hours.
- If buffered > upper bound $\Rightarrow$ log a warning. Planning still runs and returns the best achievable schedule.

6. Tunable Parameters

- **confidence_floor** (default 0.95, range 0.5–1.0) — demand is divided by this value, so a *lower* floor means a *larger* buffer. A floor of 0.80 plans for $1/0.80 = 1.25\times$ the required energy — target almost certainly met even with optimistic forecasts, but $\sim 25\%$ more spend. A floor of 1.00 plans exactly the required energy — cheapest, but any forecast slip undershoots the target.
- **confidence_exponent** (default 0.5, range 0.0–1.0) — exponent applied to per-hour confidence in the ranking divisor ($\text{raw_cost} / \text{conf}^e$). $e = 1$ is the hard penalty (a conf-0.7 hour pays a 43% markup); $e = 0.5$ softens it to $\sim 20\%$, so price differences dominate more often; $e = 0$ ignores confidence in ranking entirely. Affects *which* hour is picked, not how much energy is planned.
- **value_of_full** (default 0.0) — raw-cost threshold for opportunistic top-up after the target is hit. Raise it to keep buying while prices dip below the threshold (free top-ups when grid is cheap); leave at 0 to stop exactly at target.
- **solar_is_free** (default True) — prices solar at 0 to maximize self-consumption. Set False to price it at feed_in.
- **feed_in** (default 0.08) — only used when solar_is_free=False. Higher = solar looks expensive and loses to cheap grid hours; lower = solar wins more often.

7. Running the Project

```
poetry install
poetry run app -i data/example_request.json -o result.json -v
poetry run pytest
```

7.1 Web UI (Streamlit) via Docker

A small Streamlit app at `web/streamlit_app.py` wraps the same `run()` pipeline behind a browser UI: sidebar inputs for the vehicle and tuning parameters, a forecast pasted as JSON or uploaded as CSV, and a result panel showing the per-hour plan, total cost, cost per kWh, and a feasibility check.

```
docker build -t ev-scheduler .
docker run --rm -p 8501:8501 ev-scheduler
# open http://localhost:8501
```

The CSV upload accepts files with exactly the columns `hour`, `price`, `solar`, `confidence` (case-insensitive):

```
hour,price,solar,confidence
6:00,0.32,0.0,1.00
7:00,0.28,0.0,1.00
8:00,0.25,0.2,0.95
```

7.2 As a Library or FastAPI Route

As a library or FastAPI route:

```
from fastapi import FastAPI
from app.main import run

app = FastAPI()

@app.post("/schedule")
def schedule(payload: dict):
    return run(payload)
```

8. Engineering Properties

The pipeline is built as small, single-purpose modules connected through narrow interfaces. This keeps the code **loosely coupled, modular, testable, and easy to extend** — each stage can be unit-tested in isolation and swapped without touching the others.

8.1 Handling Multiple Data Sources and Algorithm Variants

In a real deployment, forecasts may come from a CSV, API, database, or internal feed, while the planner could use a greedy approach today and a linear programming model tomorrow. The **Strategy pattern behind a thin interface** cleanly accommodates this variability by allowing components to be swapped without affecting the rest of the system.

```
from typing import Protocol

class ForecastSource(Protocol):
    def load(self) -> list[Hour]: ...

class Planner(Protocol):
    def plan_charging(self, ...) -> list[dict]: ...
```

`main.run` depends on the protocol, not the concrete class. A small **Factory** picks the variant from config; a **Decorator** layer can add logging, caching, or retries without touching the algorithm. The result:

- New data source = one new class implementing **ForecastSource** — no edits anywhere else (open/-closed).
- New algorithm = one new class implementing **Planner** — swap via config (Liskov + dependency inversion).
- Each variant is unit-testable with a fake source or planner; the boundary Pydantic models keep the algorithm input predictable and deterministic regardless of where the data came from.

9. Scaling to High Request Volume

Since the core is already stateless (`dict` $\rightarrow$ `dict`), further efficiency gains mainly come from optimizing request dispatching and shared-work caching.

9.1 Path to Many Requests / Second

- **Stateless workers, horizontal scaling** — run the FastAPI app behind a load balancer; add Gunicorn / Uvicorn workers per CPU. No coordination needed.
- **Async queue for bursts** — accept requests on a fast endpoint, enqueue to Redis / RabbitMQ, drain with a worker pool. Smooths spikes and gives natural back-pressure.
- **Forecast cache** — requests sharing the same forecast (fleet of vehicles, same region) reuse a cached tier list keyed by forecast hash.
- **Batch endpoint** — one HTTP call plans for many vehicles against a shared forecast, amortizing parsing and tier construction.
- **Observability** — per-stage latency (validation, feasibility, planning), request counts, and the infeasibility rate to detect performance issues.
- **Container + autoscaling** — ship the Docker image, deploy on Kubernetes / ECS with HPA on CPU or request latency.

10. Project Layout

app/	
main.py	run() pipeline + CLI entry
model/models.py	Pydantic models
service/planner_service.py	Tier-based greedy scheduler
utils/cli.py	argparse + JSON I/O
utils/data_validator.py	Input validation
utils/feasibility_checker.py	Feasibility report
data/example_request.json	Sample input
web/streamlit_app.py	Web UI
tests/	Unit + integration tests
Dockerfile	Container build (optional)